Verifier-Gated LLM NetOps: A Preregistered Artifact-Backed Evaluation of Input Sanitization and Deterministic Verification

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a preregistered paired confirmatory experiment (study_id cand-2026-03) that measured whether template-based input sanitization combined with deterministic verifier-gating (and at-most-one automated repair) changes the rate at which a deterministic validator accepts LLM-generated CLI configurations under adversarially mutated intents. The frozen run used the declared generator gpt-5-mini and deterministic_netops_validator_v5 within an orchestration adapter. Planned episodes = 300; completed episodes = 282; complete paired outcomes used in the primary analysis = 141. Observed per-pair acceptance: Baseline 135/141 (95.7447%); Guarded 141/141 (100.0%); paired absolute difference = 0.04255319148936165; exact two-sided McNemar p = 0.03125; 95% bootstrap percentile CI (10,000 resamples, seed = 1729) = [0.014184397163120567, 0.07801418439716312]. We reconcile preregistration wording vs the formal estimand, document pipeline semantics and shared_initial_candidate reuse, disclose a protocol deviation (unset runtime sampling temperature), and discuss operational interpretation and limitations constrained by synthetic scope and single-model/validator evaluation. All principal numeric claims are supported by the archived execution and analysis artifacts (execution manifest [14]).**

I. Introduction

Generative LLMs are increasingly proposed to translate high-level operator intent into device CLI, promising automation gains for NetOps. However, operational risks remain when generated outputs violate sequencing constraints, CLI grammar, or safety invariants; adversarially crafted inputs can exacerbate these risks. Prior prototype studies show that coupling LLM generation with deterministic verification and targeted repair can reduce observable errors, but existing evaluations often rely on token- or reference-match metrics that do not directly measure deployability or acceptance by a deterministic validator [3,8,9].

This work asks a narrowly scoped, operational question amenable to preregistered inference: How effective are template-based input sanitization and deterministic verifier-gating (with at-most-one automated repair) at changing the rate at which a deterministic validator accepts LLM-generated CLI configurations when inputs are adversarially mutated? The study cand-2026-03 preregistered a single formal estimand (paired_success_rate_difference_guarded_minus_baseline) and a confirmatory paired test. Because the registered generation semantics were shared_initial_candidate, the paired comparison isolates downstream guarded-pipeline effects (sanitization, gating, permitted repair, and final validation) applied to a single shared LLM draft per intent rather than differences in initial generation.

Contributions. (1) A preregistered, artifact-backed paired experiment executed end-to-end and reconciled against the archived preregistration; (2) a precise, artifact-grounded description of per-condition pipeline semantics that demonstrates shared_initial_candidate reuse; (3) quantitative confirmatory results showing a small but statistically detectable absolute change in deterministic-validator acceptance under the guarded pipeline on a synthetic adversarial-intent bank; and (4) explicit reconciliation of preregistration natural-language hypothesis wording with the formal estimand and disclosure of a protocol deviation (unset sampling temperature) together with reproducibility guidance.

II. Related Work

Deterministic network-verification tools. Formal deterministic validators such as Header Space Analysis and VeriFlow (and related NetPlumber-style approaches) are widely used to encode and rapidly check forwarding, access-control, and sequencing invariants before deployment [5–7]. These tools operate over explicit, analyzable models of packet headers or configuration effects and therefore report property-level violations (reachability, isolation, precedence) rather than syntactic differences. That representational difference explains their suitability as deployment gates in LLM-assisted NetOps: validators can accept or reject a candidate configuration with a semantics-focused criterion, but they require explicit formalization of the intended invariants and do not automatically cover every dynamic or protocol-level safety property [5–7,14].

LLM-assisted configuration and evaluation gaps. Recent empirical work studying LLMs for NetOps (e.g., NetConfEval and NetLLMBench) consistently finds that model-only generation produces both syntactic errors and higher-level semantic mistakes (ordering, precondition violations, protected-interface breaches) when translating intent to CLI [3,8,9]. Many published evaluations focus on string- or reference-match metrics that measure surface similarity to a gold command sequence; such metrics correlate poorly with deployability because they neither verify execution-order constraints nor check reachability/forwarding invariants. Surveys of AIOps/telecommunications emphasize fragmented evaluation practices and call for operationally grounded benchmarks

that measure deployability, safety, and validator acceptance rather than token overlap alone [4,9].

Generate–validate–repair architectures. A common mitigation pattern is the generate–validate–repair architecture: an LLM produces a candidate, an automated verifier checks it, and feedback is converted into constrained repair prompts or corrective actions that produce revised candidates. Prototype studies and orchestration approaches demonstrate that integrating external checks into the generation loop improves concrete outcome metrics on narrow tasks, provided verifier feedback is sufficiently precise and repair scope is constrained [11,3]. These demonstrations show promise but reveal dependencies: repair effectiveness depends on the granularity of verifier diagnostics and the boundedness of repair policies (limited number of automatic repairs and explicit constraints on allowable edits).

Security and adversarial inputs. The LLM-systems security literature documents prompt-injection and indirect prompt-attacks that can subvert downstream behavior when external or user-supplied text is trusted by an orchestrator [10]. These attack modes motivate operational defenses used in practice, such as template-based input sanitization, slot validation, and gating designs that deterministically block candidates failing explicit safety checks. Defensive evaluations are often performed on synthetic or constrained datasets and may not exhaustively explore multi-source pipelines, motivating focused NetOps-specific threat-modeling and mitigation studies.

Benchmarks and measurement desiderata. Several benchmark efforts attempt to exercise common NetOps intents but differ in goals and coverage. A recurring limitation is the lack of standardized, validator-centered deployability metrics. This paper situates itself against those gaps by preregistering an estimand that uses deterministic-validator acceptance as the primary quantity of interest and by explicitly using shared_initial_candidate semantics so that post-generation pipeline elements (sanitization, gating, bounded repair) are the causal factors under test.

III. Methodology

Preregistration and estimand. The study cand-2026-03 was preregistered prior to observing outcomes (preregistration archived; see execution manifest [14]). The primary estimand is paired_success_rate_difference_guarded_minus_baseline: for each adversarial intent we define a binary outcome equal to 1 if the deterministic validator would accept the final candidate for deployment and 0 otherwise. The preregistered confirmatory test is an exact two-sided McNemar on paired outcomes. Bootstrap percentile 95% CIs use 10,000 resamples with seed = 1729.

Design and task bank. The preregistered design targeted N = 150 adversarial intents (paired episodes across Baseline and Guarded). The synthetic intent templates exercise interface admin changes, MTU and VLAN updates, constrained required sequences (e.g., shutdown–change–restore patterns), and protected-interface rules. The adversary model applied deterministic mutation heuristics to templates; generation seeds and mutation taxonomy are archived in the execution manifest. Scope is limited to these synthetic templates and the deterministic CLI grammar encoded in our validator; no private production data was used.

Model, validator, and orchestration. The declared generator is gpt-5-mini and the deterministic validator is deterministic_netops_validator_v5. Orchestration used a hosted generate–validate–repair adapter implementing the guarded pipeline. The guarded branch applied template-based sanitization prior to gating; the sanitizer and gating policy are described in the archived task payloads referenced by the execution manifest [14]. The archived model configuration recorded declared_model_version = "gpt-5-mini" and an unset temperature field (temperature = null); this unset sampling parameter is disclosed as a protocol deviation (see Disclosure Statement).

Pipeline timeline and shared_initial_candidate semantics. Because generation semantics were registered as shared_initial_candidate, the execution followed this per-pair timeline: (1) Task instantiation and adversarial mutation; (2) Shared initial generation: a single LLM call produced the shared_initial_candidate which was used for both Baseline and Guarded scoring; (3) Baseline scoring: deterministic validator applied to the shared_initial_candidate; (4) Guarded branch: sanitizer applied and, if permitted by policy, at-most-one automated repair LLM call was allowed; (5) Final validation: the validator re-evaluated the sanitized or repaired candidate and recorded the final guarded decision. Per-episode validator traces, provider-call accounting, and task manifests supporting these steps are archived and cited via the execution manifest [14].

Missingness, exclusions, and analysis plan. The preregistered missingness_plan specified excluding a paired unit only when both paired runs were technical failures. The archive reports completed_episodes = 282, failed_episodes = 18, and complete_pairs_included = 141. Provider-call accounting and stage counts (shared_initial_generation and repair-stage counts) are available in the archived reconciliation artifacts cited below [14]. Primary analysis used paired_binary_analysis_v1 (exact McNemar); bootstrap CI parameters and seeds are recorded in the analysis artifacts archived with the execution manifest [14]. Secondary and sensitivity analyses followed the preregistered plan described in the archived preregistration.

IV. Results

Execution and raw counts. The archived execution completed 282 of the planned 300 episodes; 18 episodes failed for technical reasons recorded per the preregistered missingness rules. After applying the exclusion rule (exclude a paired unit only when both paired runs were technical failures) 141 complete pairs entered the primary confirmatory analysis. Provider-call accounting shows hosted_model_call_event_count = 156 total model-call events, with 147 completed and 9 failed calls; stage counts report

150 shared_initial_generation events and 6 repair-stage events (reconciliation artifacts cited in [14]).

Condition-level outcomes. Condition summary reports baseline_successes = 135 out of 141 complete pairs (baseline success rate = 0.9574468085106383) and guarded_successes = 141 out of 141 (guarded success rate = 1.0). These per-condition margins are the marginal rates of validator-accepted final candidates after each per-condition pipeline executed (baseline = validator applied to the shared_initial_candidate; guarded = sanitizer $\pm$ permitted repair applied then validator re-evaluation).

Paired contingency and inference. The paired contingency table records n11 = 135 (both accepted), n10 = 6 (guarded accepted, baseline rejected), n01 = 0 (baseline accepted, guarded rejected), and n00 = 0 (both rejected). The preregistered confirmatory test (exact two-sided McNemar) yields p = 0.03125. Observed paired point estimate (guarded - baseline) = 0.04255319148936165. The 95% bootstrap percentile confidence interval for the paired difference was computed with 10,000 resamples and seed = 1729 and is [0.014184397163120567, 0.07801418439716312]. All numeric results and test parameters are archived and reproducible from the analysis artifacts cited in the execution manifest [14].

Discordant-case diagnostics and repair association. The six n10 discordant pairs (guarded=1, baseline=0) are traceable in the archived per-episode validator traces. Reconciliation links repair-stage activity to these discordant pairs: the repair stage count in the reconciliation artifact equals six, and each discordant pair shows that sanitizer normalization and/or a single permitted repair converted a draft the validator would reject into a final candidate the validator accepted. Detailed per-episode traces are retained in the archive cited below [14].

Sensitivity to technical failures. Nine paired units were excluded because both runs failed technically. As a conservative sensitivity check (not the preregistered primary analysis), treating those excluded pairs as failures yields baseline acceptance = 135/150 = 0.9000 and guarded acceptance = 141/150 = 0.9400. This sensitivity illustrates that technical failures reduce effective sample size but do not invert the observed paired difference under this conservative assumption. Failure counts and the conservative recalculation are reproducible from the archived analysis artifacts [14].

Unexecuted preregistered items. The preregistered benign-control (M = 50) and the false-positive-blocking secondary estimand were not executed in this run; no benign-control artifacts are present in the archive. Consequently no empirical false-positive-blocking rates are reported here; this omission and its practical implications are documented in the preregistration and in the archived execution manifest cited below [14].

V. Discussion

Hypothesis reconciliation and interpretive boundary. The preregistration natural-language H1 asserted that verifier-gating would reduce attacker success (fewer unsafe configurations accepted). The registered formal estimand measured paired validator acceptance-for-deployment (guarded - baseline). These constructs differ: an increase in validator acceptance can reflect successful repair of malformed but benign drafts and not necessarily reduced attacker success. Our observed guarded$\rightarrow$higher acceptance ($\approx$4.255 percentage points) occurred because sanitizer and permitted repair recovered reparable drafts that the baseline validator rejected. Therefore the preregistered directional statement about reducing attacker success is not supported when interpreted as an attacker-centric safety reduction; we therefore report and interpret the archived formal estimand explicitly and caution against conflating validator acceptance with attacker-centric safety without an independent semantic-safety oracle.

Operational implications and repair–gate co-design. Within the synthetic setting, input sanitization plus bounded automated repair can reduce operator rework by converting invalid drafts into validator-accepted candidates. However, this benefit depends on the validator's semantic coverage and the scope of permitted repairs: the six discordant cases show repairs enabling acceptance, which is beneficial if repairs preserve intended semantics but risky if repairs mask adversarial manipulations. This highlights an operational co-design requirement: gating policies, repair scope, and validator expressiveness must be jointly specified to preserve safety in deployment.

Threats to validity. Internal validity: 18 technical failures reduced the effective sample and triggered the preregistered exclusion rule. Construct validity: the deterministic validator encodes CLI grammar, required sequences, and protected-interface invariants but does not capture all dynamic protocol-level semantics; acceptance-for-deployment is therefore a partial proxy for operational safety. External validity: a single generator (gpt-5-mini), a single validator, and a synthetic intent bank limit generalization across vendors, CLI dialects, and production traffic. Conclusion validity: the absolute effect is small against a high baseline (ceiling effect); statistical detectability does not imply broad practical impact.

Reproducibility guidance and protocol deviation. The archived model configuration recorded temperature = null (unset). We disclose this as a protocol deviation because provider-side defaults can affect sampling. To preserve paired comparability despite the unset parameter, the pipeline employed shared_initial_candidate semantics (single generation per pair) and provider-call accounting documented in the reconciliation artifact. For deterministic reproduction we recommend explicitly setting temperature = 0 and re-executing the archived execution manifest; the manifest and analysis artifacts contain the complete run record and parameters necessary for reproduction and verification (archive cited as [14]).

Recommendations. (1) Execute the preregistered benign-control to measure false-positive blocking; (2) preregister an attacker-centric estimand that couples validator acceptance with an independent semantic-safety oracle or reachability tests; (3) diversify generators and validators across vendors and CLI dialects; and (4) fix sampling settings (temperature = 0) for deterministic reproduction. Each recommendation follows from the archived evidence and provider-call accounting in the

execution manifest [14].

VI. Limitations

- Synthetic task bank and intent templates — not real production traffic or operator logs; results may not generalize to field datasets.
- Single-model evaluation (gpt-5-mini) and single validator implementation limit external validity across vendors, protocols, and model families.
- Validator coverage constrained to encoded safety properties (CLI grammar, required sequences, protected-interface invariants); some protocol-level or dynamic runtime semantics are not represented.
- Technical failures (18/300 episodes) reduced effective sample size; paired exclusion (both-run failures = 9) was preregistered and applied.
- Adversary model limited to mutations of synthetic intents; prompt-injection in retrieval-augmented or multi-source pipelines was not exhaustively evaluated.
- A preregistration natural-language hypothesis phrasing differed from the registered formal estimand; results are reported and interpreted with respect to the archived formal estimand.
- The preregistered benign-control (M = 50) and false-positive-blocking secondary estimand were not executed; therefore no empirical false-positive-blocking claims are made.

VII. Conclusion

A preregistered, artifact-backed paired experiment on a synthetic adversarial intent bank found that template-based input sanitization combined with deterministic verifier-gating and at-most-one automated repair produced a small but statistically detectable absolute increase in deterministic-validator-accepted LLM-generated configurations under the archived formal estimand (paired difference ≈ 0.04255; $p = 0.03125$; 95% CI [0.01418, 0.07801]). Diagnostics show the six discordant pairs were associated with permitted repair calls that enabled acceptance. Importantly, this formal-estimand improvement does not equate to measured reduction in attacker success under attacker-centric definitions. The run is limited by synthetic scope, a single model/validator, and a protocol deviation (unset runtime temperature). Future work should execute the preregistered benign-control, measure attacker-centric estimands with an independent semantic-safety oracle, diversify models/validators, and set explicit sampling parameters for reproducible deterministic runs. All principal numbers and reconciliation claims above are supported by archived execution and analysis artifacts; the execution manifest is the primary machine-verifiable archive and is cited below as [14].

Disclosure Statement

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: a human Research Director selected the topic family (Generative AI / LLMs for NetOps) and constrained the initial master prompt prior to the autonomy lock; all literature review, preregistration, study selection, code generation, experiment execution, analysis, peer-review cycles, manuscript drafting, and revision reported here were performed autonomously after the lock. Declared models/tools and orchestration: primary generator = gpt-5-mini; deterministic validator = deterministic_netops_validator_v5; task generator = deterministic_netops_task_generator_v5; orchestration adapter family = hosted_netops_gvr_v1. Preregistration: study cand-2026-03 was preregistered and archived prior to observing outcomes (preregistration archived; see execution manifest [14]). Protocol deviation: the archived run recorded an unset runtime sampling temperature (temperature = null in the model configuration); this is disclosed because provider-side defaults may affect sampling determinism. The execution manifest is the primary machine-verifiable archive for execution, analysis, and artifact-level provenance and is cited here as [14]. No human manually edited or corrected the manuscript technical content after the autonomy lock.

References

[1] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659
[2] https://doi.org/10.1145/2342441.2342452
[3] https://doi.org/10.1145/3626111.3628194
[4] https://doi.org/10.1145/3605764.3623985
[5] http://arxiv.org/abs/2302.04761
[6] https://doi.org/10.1145/3656296
[7] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
[8] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
[9] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
[10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023, 10.1145/3605764.3623985
[11] Yupeng Chang, Xu Wang, Jindong Wang, Yuan-Hsuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, Xing Xie, "A Survey on Evaluation of Large Language Models", 2023, 10.48550/arxiv.2307.03109
[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Răileanu, María Lomelí, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools", 2023, 10.48550/arxiv.2302.04761
[13] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
[14] execution_manifest